

Introducción a la Arquitectura de Software

Contenidos

- ¿Qué es Arquitectura de Software?
- Atributos de Calidad
- Estilos arquitectónicos
- Pensando en términos de Arquitectura
- Buenas prácticas en Arquitectura

¿Qué es la Arquitectura de Software?



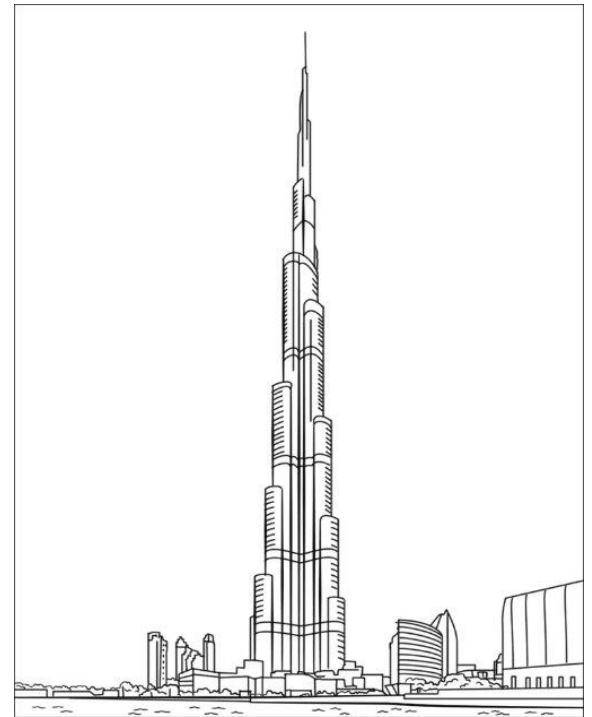
Arquitectura

- Abstracción de alto nivel de un sistema de software



Abstracción

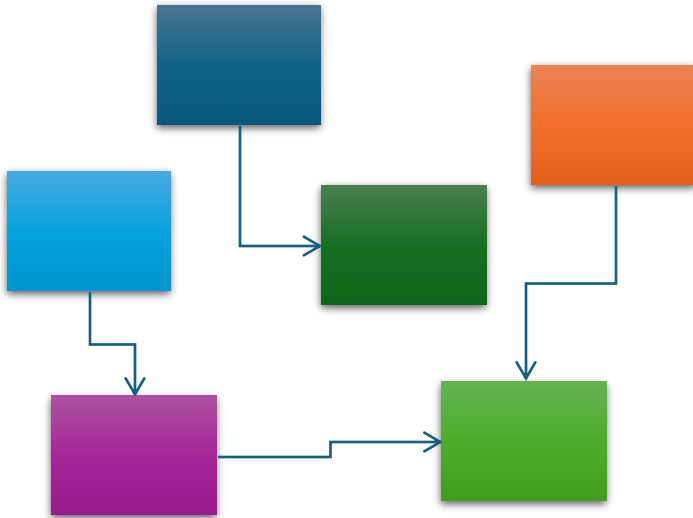
- Omitir detalles innecesarios
- Enfocarse solo en lo esencial



Estructura + Comportamiento

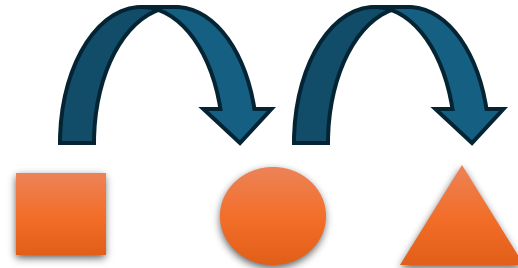
- Estructura

- Componentes
- Propiedades / Atributos
- Conexiones

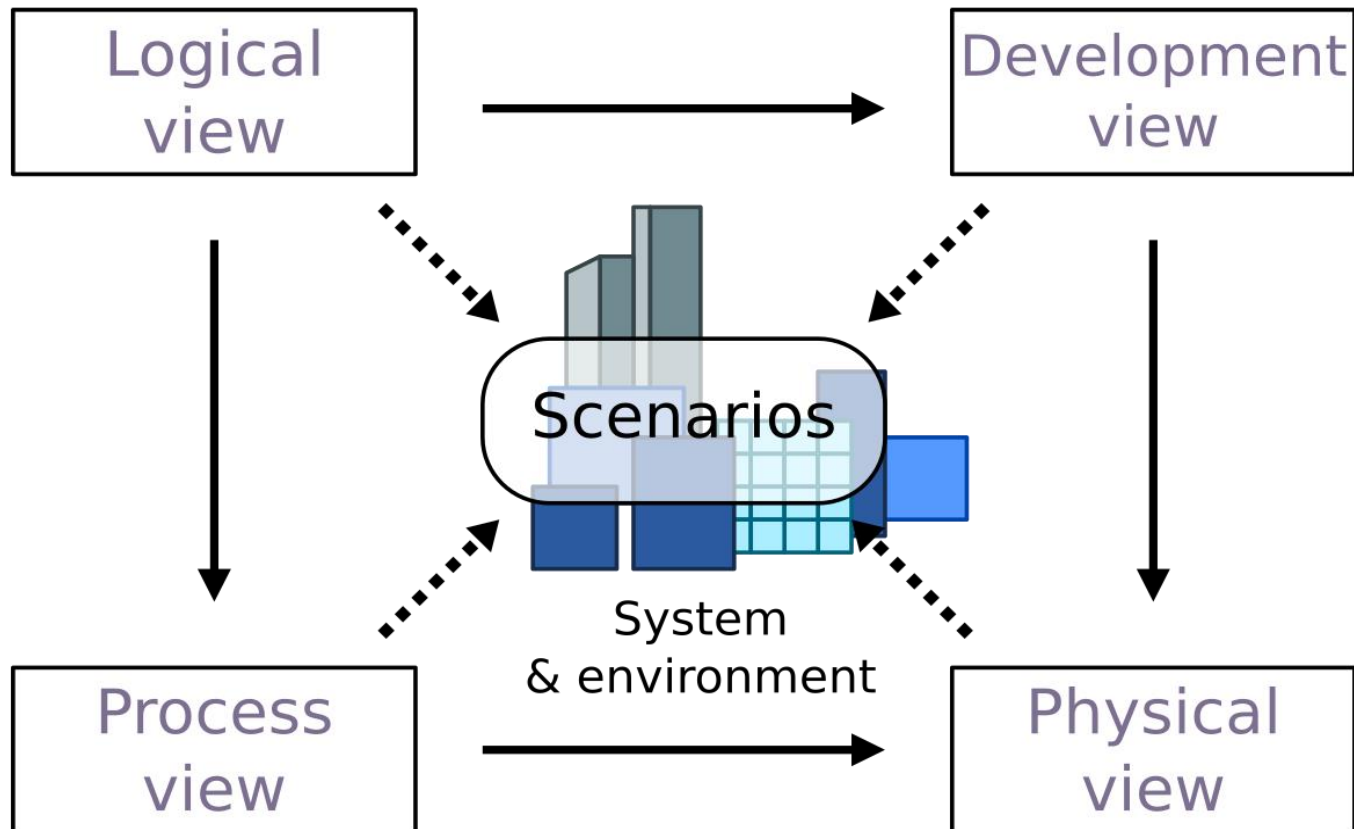


- Comportamiento

- Cómo cambia en el tiempo



Vistas arquitectónicas (Ej. Modelo 4+1)



Recomendaciones

- Equipo de arquitectura pequeño
- Priorizar atributos de calidad
 - Decisiones arquitectónicas
 - Trade-offs
- Documentación
 - Incremental
 - Usar Vistas
- Evaluación temprana
 - Satisfacción de atributos de calidad
 - Trade-offs
- La arquitectura debe permitir implementación incremental
 - "Esqueleto"
 - "Carne"

¿Cuál es la diferencia entre
Arquitectura y Diseño
detallado?



Decisiones e impacto

Decisiones arquitectónicas

- Largo plazo
- Impacto general
- Cambios requieren mayor esfuerzo
- Estratégica

Decisiones de Diseño detallado

- Corto plazo
- Impacto acotado
- Cambios requieren menor esfuerzo
- Táctica

¿Ejemplos?



Atributos de calidad

Availability

Deployability

Energy
efficiency

Integrability

Modifiability

Performance

Safety

Security

Testability

Usability

Availability

- El sistema está preparado para responder a una petición en el momento requerido
- Availability
 - Reliability
 - Recovery
 - Fault-tolerance
- Fallos (Faults)
 - Prevención + Detección + Recuperación

Deployability

- ¿Qué tan fácil es poner en producción una nueva versión del sistema?
- Proceso de instalación
- Pasos manuales vs automáticos
- Tiempo requerido
- CI/CD
- Ambientes
 - Desarrollo
 - Pruebas
 - Producción

Energy Efficiency

- ¿Cuánta energía requirere el funcionamiento del sistema?
- Escenarios bajo los cuales el consumo varía
- Cómo afecta otros atributos
 - Performance
 - Availability
 - Modifiability

Integrability

- Capacidad de integrarse con otros sistemas
- Interoperabilidad
- Qué tan fácil es de adaptar para nuevas integraciones
- Cuánto cuesta la integración

Modifiability

- Capacidad para ser cambiado
 - ¿Qué se puede cambiar?
 - ¿Qué tan probable es que requiera cambios?
 - ¿Cuándo se cambia?
 - ¿Quién lo cambia?
 - ¿Cuánto cuesta?
- Mantenibilidad
- Escalabilidad
- Portabilidad

Performance

- Capacidad de cumplir con requisitos de tiempo
 - Velocidad
 - Uso de recursos (CPU, memoria, etc.)
 - Tiempo de respuesta

Safety

- "No mates a nadie"
- Criticidad

Criticidad ("defects cause loss of...")

Life (L)	L6	L20	L40	L100	...
Essential Money (E)	E6	E20	E40	E100	
Discretionary Money (D)	D6	D20	D40	D100	
Comfort (C)	C6	C20	C40	C100	
	1-6	20	40	100	Cantidad de stakeholders

Security

- Proteger la información de uso no autorizado
- Proveer acceso a los que sí estén autorizados
- Conceptos
 - Confidentiality: protección contra acceso no autorizado
 - Integrity: protección contra manipulación no autorizada
 - Availability: disponibilidad para uso legítimo

Testability

- Capacidad del software de
 - Ser probado y encontrar potenciales errores
 - Revelar errores tempranamente
 - Controlar entradas y salidas de cada componente
 - Reproducir un error encontrado previamente

Usabilidad

- Qué tan "fácil" es para un usuario realizar tareas sobre el sistema
- Soporte al usuario
- Facilidades para aprendizaje del sistema
- Eficiencia de uso
- Minimizar impacto de errores
- Adaptabilidad
- Satisfacción del usuario

Estilos Arquitectónicos / Patrones Arquitectónicos

Layered / N-Tier

Modular monolith

Pipeline / Pipe and filter

Microkernel

Service-based

Event-driven

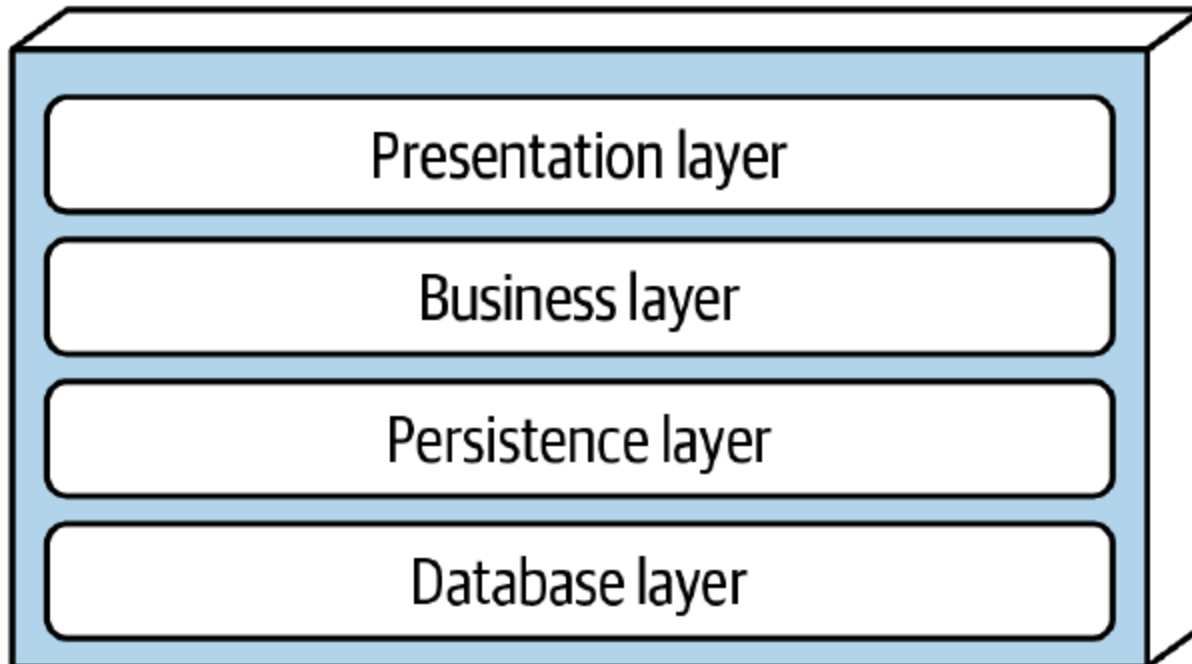
Orchestration-driven Service-Oriented

Microservices

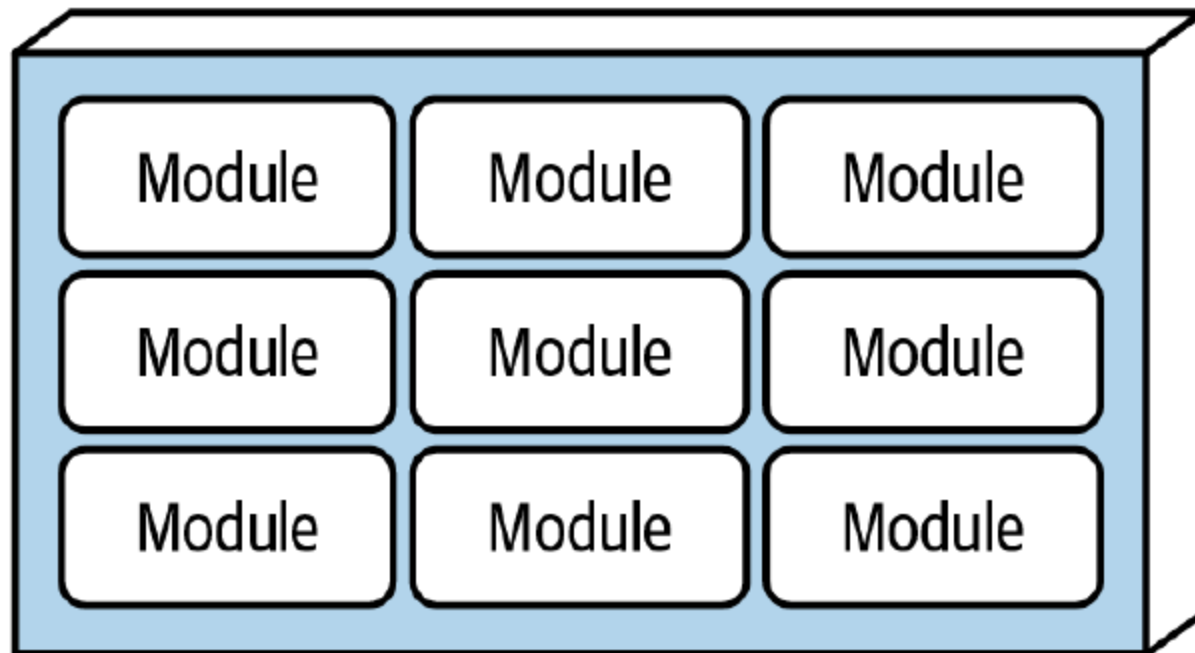
CQRS

Broker-domain

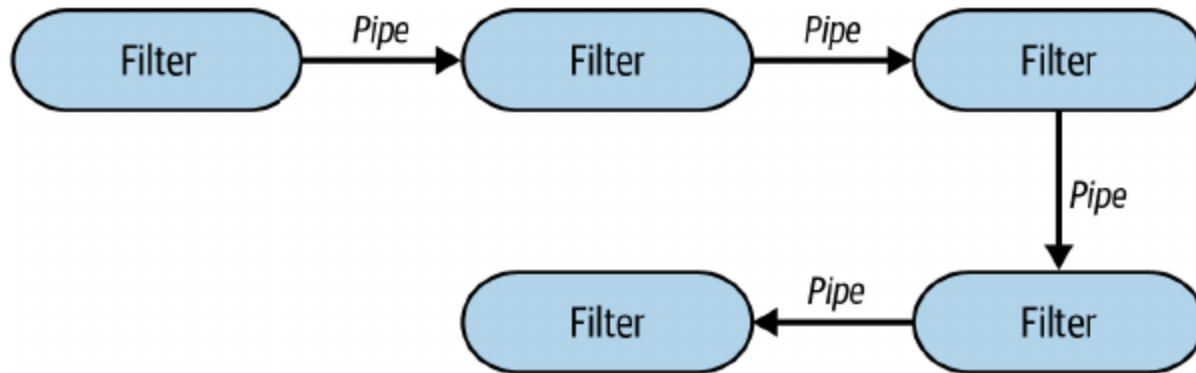
Layered / N-Tier



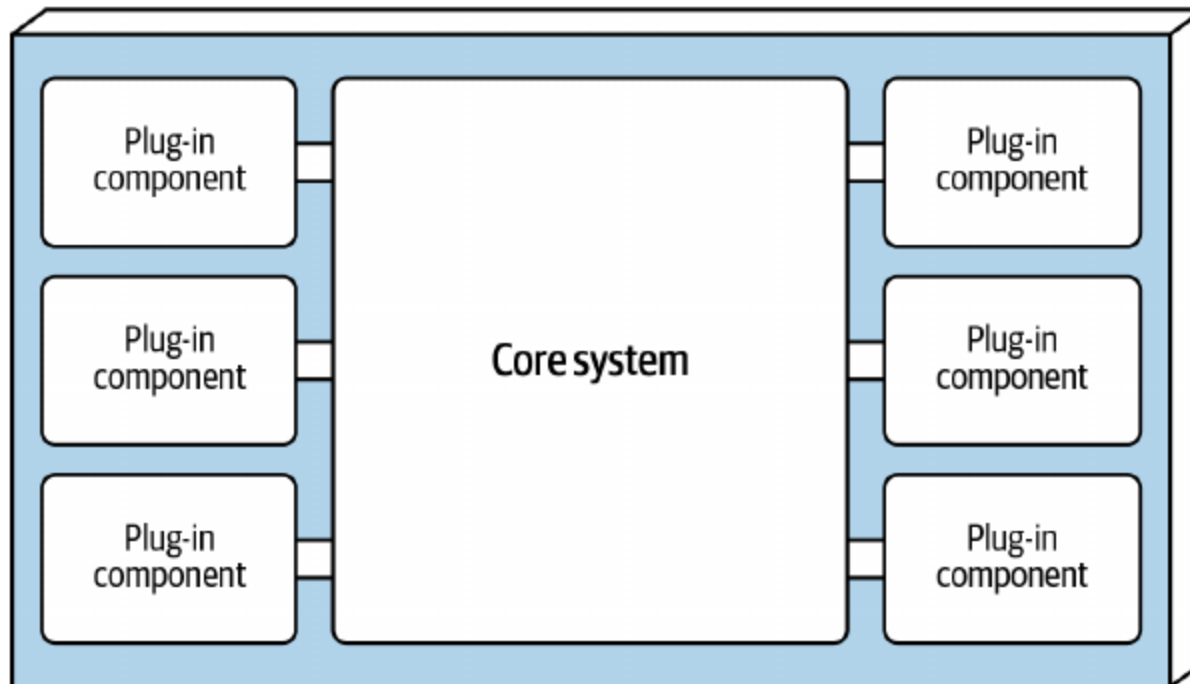
Modular Monolith



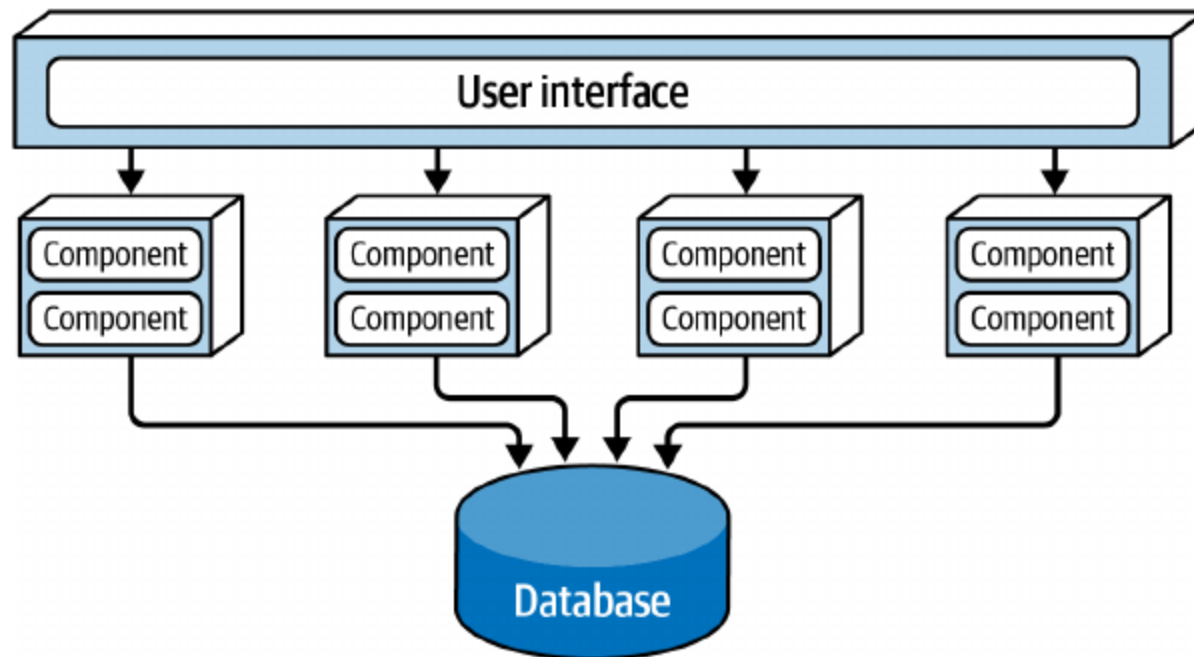
Pipeline / Pipe and Filter



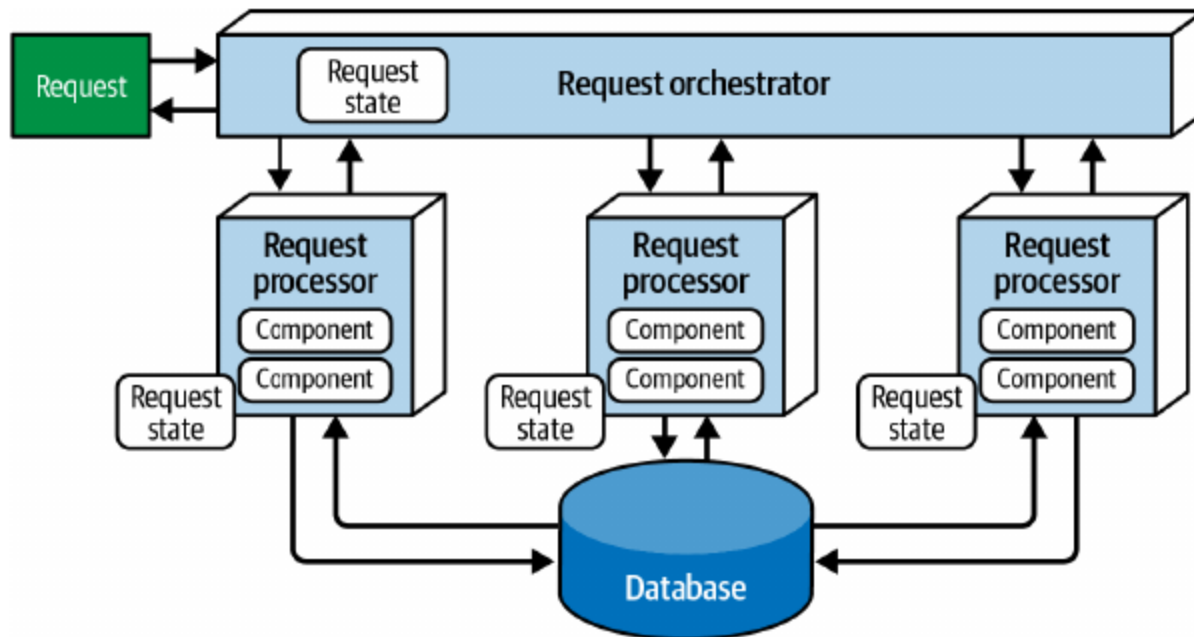
Microkernel



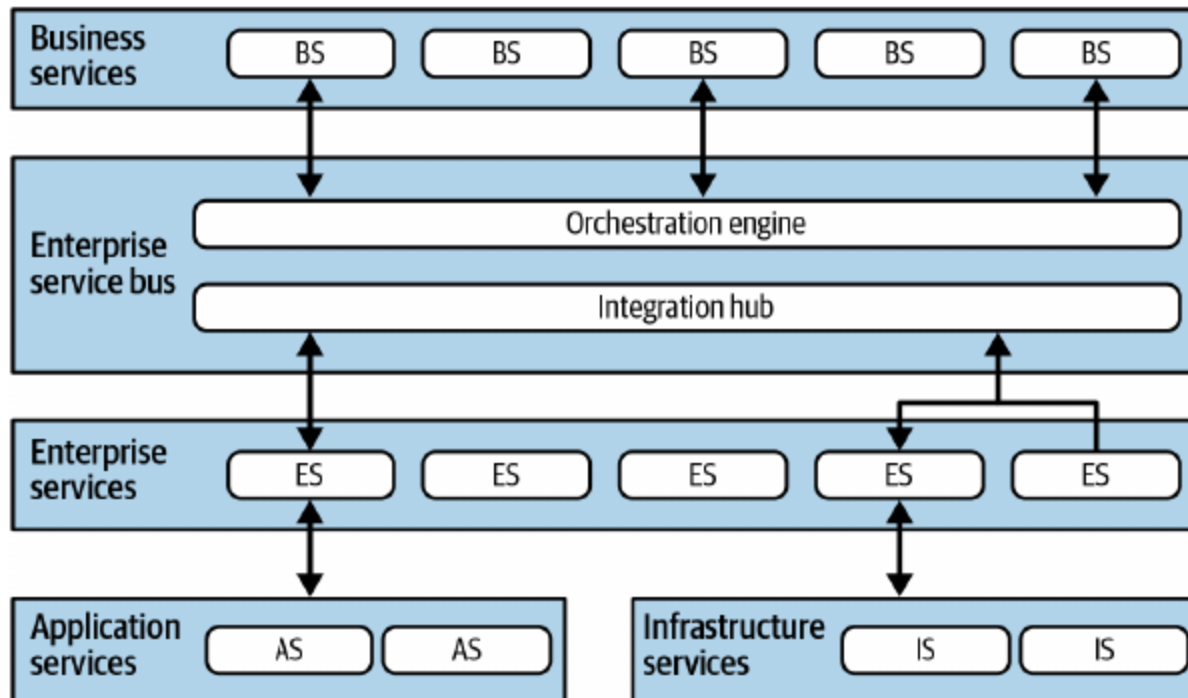
Service-based



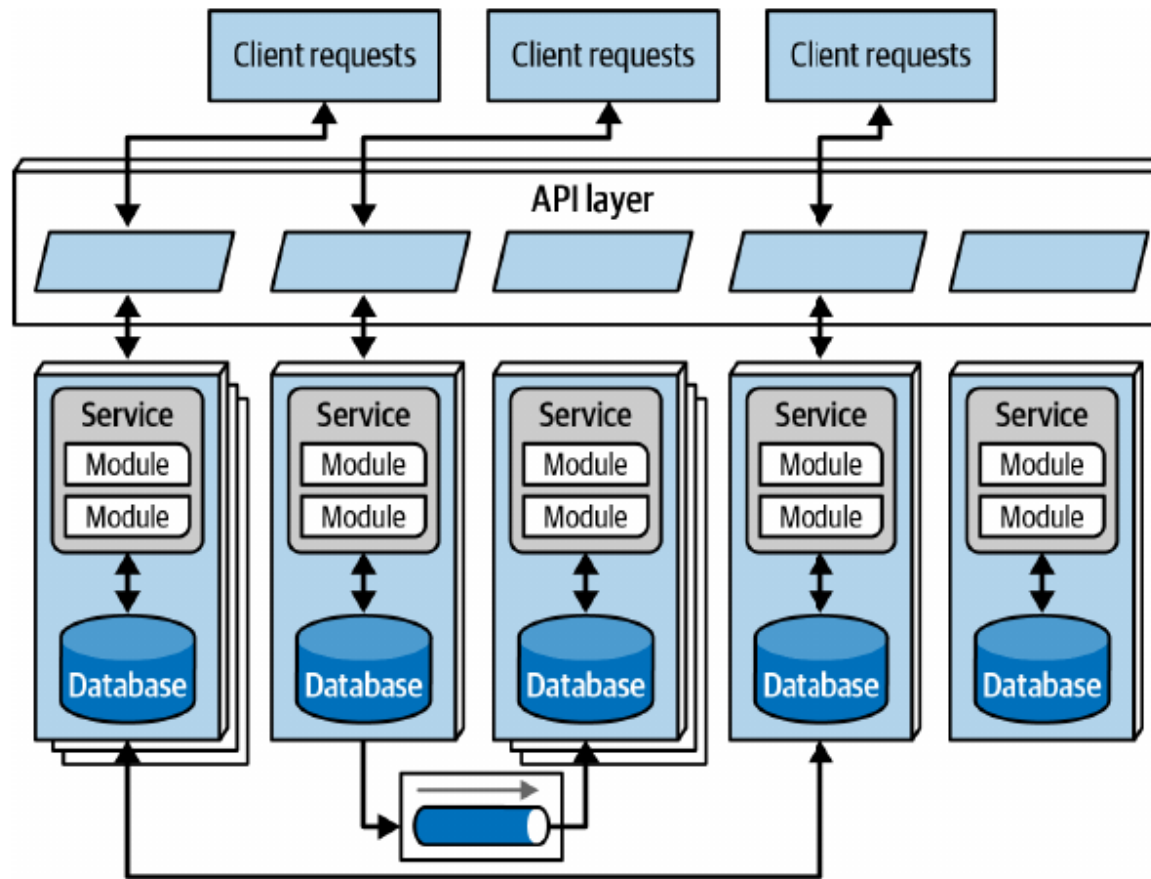
Event-driven



Orchestration-Driven Service-Oriented



Microservices



Trade-offs

- Las decisiones arquitectónicas favorecen algunos atributos de calidad a costa de otros
 - Teorema CAP "escoja solo dos de las siguientes"
 - Consistency: Todos los clientes ven la misma información
 - Availability: Todas las peticiones reciben respuesta sin errores
 - Partition tolerance: El sistema sigue funcionando a pesar de haber comunicación no confiable entre sus partes
 - Eficiencia energética
 - Eficiencia vs performance
 - Eficiencia vs availability

Resumiendo

- Arquitectura = Estructura + Comportamiento
- Vistas
 - Diferentes perspectivas
- Arquitectura vs diseño detallado
 - La diferencia está en el nivel de detalle
- Atributos de calidad
 - Dirigen las decisiones de diseño
- Estilos / Patrones arquitectónicos
 - Soluciones a los problemas recurrentes

Ejercicio – Use el conocimiento que tenga actualmente

- Para las propuestas de sistemas de software de cada equipo
 - Definir y priorizar los atributos de calidad (alta, media, baja)
 - Definir los principales componentes y conexiones
 - ¿Cómo la arquitectura propuesta satisface cada atributo de calidad?